Week 5

Relational Algebra

Join query

Relational Algebra Overview

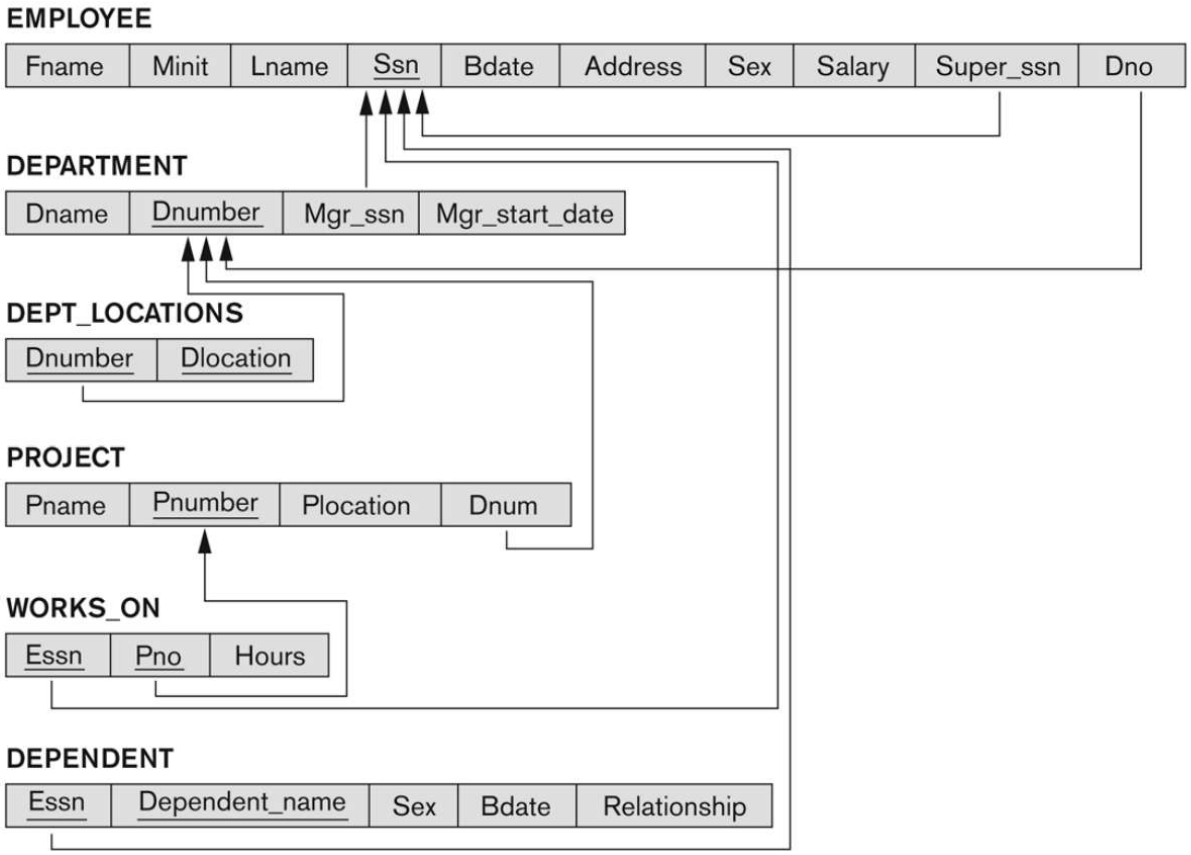
- Relational algebra is the mathematical foundation of relational databases.
- It consists of basic set of operations for the relational model which provide the logic behind SQL queries.
- The result of an operation is a new relation, which may be formed from one or more input relations.
- A sequence of relational algebra operations forms a **relational algebra expression**

Relational Algebra Overview

- Relational Algebra consists of several groups of operations
  - **Unary Relational Operations**
    - SELECT (symbol: σ (sigma)) (corresponds to Select * in MySQL)
    - PROJECT (symbol: π (pi)) (corresponds to Select column name)
  - **Binary Relational Operations**
    - JOIN (several variations of JOIN exist)
  - **Relational Algebra Operations From Set Theory**
    - UNION (∪), INTERSECTION (∩), DIFFERENCE (–) , CARTESIAN PRODUCT (x)

Database State for COMPANY

Figure 5.7
Referential integrity constraints displayed on the COMPANY relational database schema.



Unary Relational Operations: SELECT

- The **SELECT** operation (denoted by σ (sigma)) is used to select a subset of the tuples from a relation based on a **selection condition**.
- Examples:
- Select the EMPLOYEE tuples whose department number is 4:

  $\sigma_{DNO = 4}(EMPLOYEE)$

  Equivalent MySQL query = Select * from Employee where DNO=4;
- Select the employee tuples whose salary is greater than $30,000:

  $\sigma_{SALARY > 30,000}(EMPLOYEE)$

Unary Relational Operations: SELECT

- In general, the select operation is denoted by
- $\sigma_{\text{<selection condition>}}(R)$
  - the symbol σ (sigma) is used to denote the select operator
  - the $_{\text{selection condition}}$ is a Boolean (conditional) expression specified on the attributes of relation R
  - tuples that make the condition **true** are selected and appear in the result of the operation
  - tuples that make the condition **false** are filtered out and discarded from the result of the operation

Unary Relational Operations: SELECT

- **SELECT Operation Properties**
- The SELECT operation $\sigma_{<\text{selection condition}>}(R)$ produces a relation S that has the same schema (same attributes) as R
- SELECT σ is commutative: order of operations doesn't matter
  - $\sigma_{<\text{condition1}>}(\sigma_{<\text{condition2}>}(R)) = \sigma_{<\text{condition2}>}(\sigma_{<\text{condition1}>}(R))$
- SELECT operation is associative, i.e it can be grouped in any order:
  - $\sigma_{<\text{condi1}>}(\sigma_{<\text{condi2}>}(\sigma_{<\text{condi3}>}(R)) = \sigma_{<\text{condi2}>}(\sigma_{<\text{condi3}>}(\sigma_{<\text{condi1}>}(R)))$
- A cascade of SELECT operations may be replaced by a single selection with a conjunction of all the conditions:
- $\sigma_{<\text{condi1}>}(\sigma_{<\text{condi2}>}(\sigma_{<\text{condi3}>}(R)) = \sigma_{<\text{condi1}> \text{ AND } <\text{condi2}> \text{ AND } <\text{condi3}>}(R)))$

Example

Relation **Employee(EmpID, Dept, Salary, Age)**

Cascade:

$$\sigma_{Dept='IT'}(\sigma_{Salary>5000}(\sigma_{Age<30}(Employee)))$$

Equivalent single selection:

$$\sigma_{Dept='IT' \wedge Salary>5000 \wedge Age<30}(Employee)$$

Both yield the same subset of tuples.

Unary Relational Operations: PROJECT

- PROJECT Operation is denoted by π (pi)
- This operation keeps certain columns (attributes) from a relation and discards the other columns.
- PROJECT creates a vertical partitioning
- Example: To list each employee's first and last name and salary, the following is used:
  - $\pi_{LNAME, FNAME, SALARY}(EMPLOYEE)$
  - Equivalent MySQL query = Select LNAME, FNAME, SALARY from Employee

Unary Relational Operations: PROJECT

- The general form of the project operation is: $\pi_{<\text{attribute list}>}(R)$
  - π (pi) is the symbol used to represent the project operation
  - <attribute list> is the desired list of attributes from relation R.
- The project operation removes any duplicate tuples
  - This is because the result of the project operation must be a set of tuples
    - Mathematical sets do not allow duplicate elements.

Example

Relation **Student(Name, Age, Grade):**

Name	Age	Grade
Ali	20	A
Ali	20	B
Zara	22	A

Now apply:

$$\pi_{Name,Age}(Student)$$

Result:

Name	Age
Ali	20
Zara	22

Unary Relational Operations: PROJECT

- PROJECT Operation Properties
- The number of tuples in the result of projection $\pi(R)$ is always less or equal to the number of tuples in R
- If the list of attributes includes a key of R, then the number of tuples in the result of PROJECT is equal to the number of tuples in R

Examples of applying SELECT and PROJECT operations

Figure 6.1
Results of SELECT and PROJECT operations. (a) $\sigma_{(Dno=4\ \textbf{AND}\ Salary>25000)\ OR\ (Dno=5\ \textbf{AND}\ Salary>30000)}$ (EMPLOYEE). (b) $\pi_{Lname,\ Fname,\ Salary}$(EMPLOYEE). (c) $\pi_{Sex,\ Salary}$(EMPLOYEE).

(a)

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5

(b)

Lname	Fname	Salary
Smith	John	30000
Wong	Franklin	40000
Zelaya	Alicia	25000
Wallace	Jennifer	43000
Narayan	Ramesh	38000
English	Joyce	25000
Jabbar	Ahmad	25000
Borg	James	55000

(c)

Sex	Salary
M	30000
M	40000
F	25000
F	43000
M	38000
M	25000
M	55000

Binary Relational Operations: JOIN

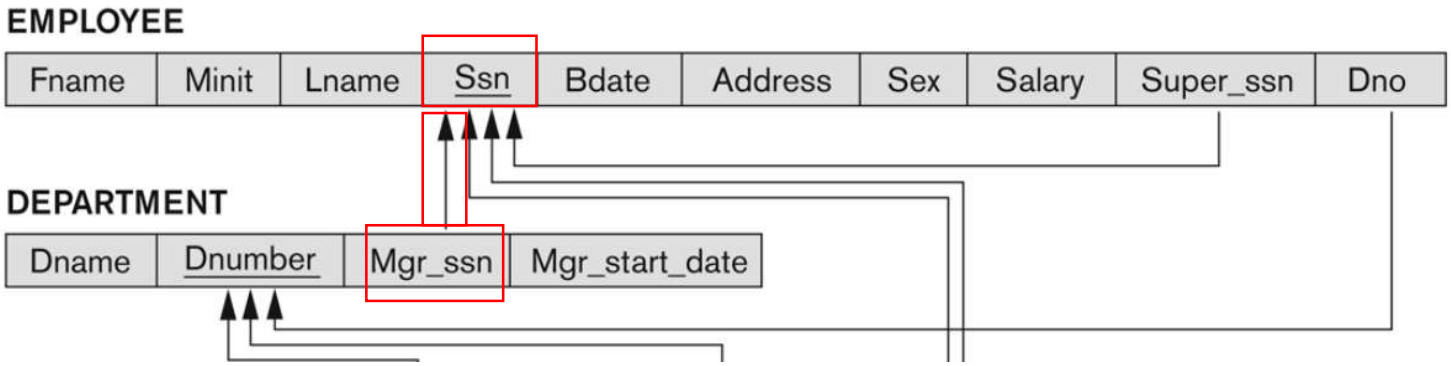
- This operation is used when we want data from more than one relation, because it allows us to combine related tuples from various relations
- The general form of a join operation on two relations R and S is:

$$R \bowtie_{\text{<join condition>}} S$$

JOIN Example

- **Example: Suppose that we want to retrieve the name of the manager of each department.**
  - To get the manager's name, we need to combine each DEPARTMENT tuple with the EMPLOYEE tuple whose SSN value matches the MGRSSN value in the department tuple.
  - We do this by using the join  operation.
  - DEPT_MGR $\leftarrow$ DEPARTMENT  $_{\text{MGRSSN=SSN}}$ EMPLOYEE
- **MGRSSN=SSN is the join condition**
  - Combines each department record with the employee who manages the department

JOIN Example



DEPT_MGR

Dname	Dnumber	Mgr_ssn	...	Fname	Minit	Lname	Ssn	...
Research	5	333445555	...	Franklin	T	Wong	333445555	...
Administration	4	987654321	...	Jennifer	S	Wallace	987654321	...
Headquarters	1	888665555	...	James	E	Borg	888665555	...

Figure 6.6
Result of the JOIN operation

Equivalent MySQL query

- SELECT * FROM DEPARTMENT JOIN EMPLOYEE ON Mgr_ssn = Ssn;

- Better way:
- SELECT * FROM DEPARTMENT JOIN EMPLOYEE ON DEPARTMENT.Mgr_ssn = EMPLOYEE.Ssn;

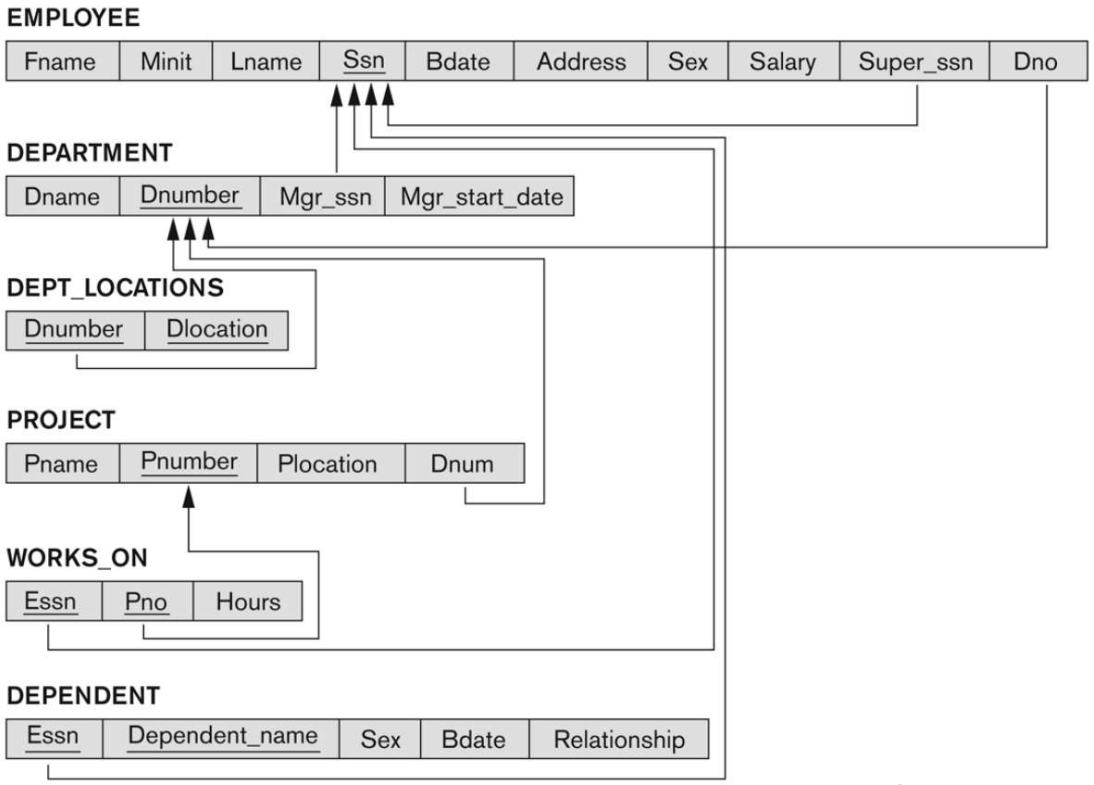
- To make Join condition cleaner and shorter give nicknames to tables:
- SELECT * FROM DEPARTMENT D JOIN EMPLOYEE E ON D.Mgr_ssn = E.Ssn;

- If you want to retrieve only Dname and manager name:

- SELECT D.Dname, E.Fname, E.Minit, E.Lname FROM DEPARTMENT D JOIN EMPLOYEE E ON D.Mgr_ssn = E.Ssn;

Question 1

Q1:Retrieve the name of the Department and the name of projects they handle?



Solution

- SELECT D.Dname, P.Pname FROM DEPARTMENT D JOIN PROJECT P ON P.Dnum = D.Dnumber

- OR

- SELECT D.Dname, P.Pname FROM PROJECT P JOIN DEPARTMENT D ON P.Dnum = D.Dnumber

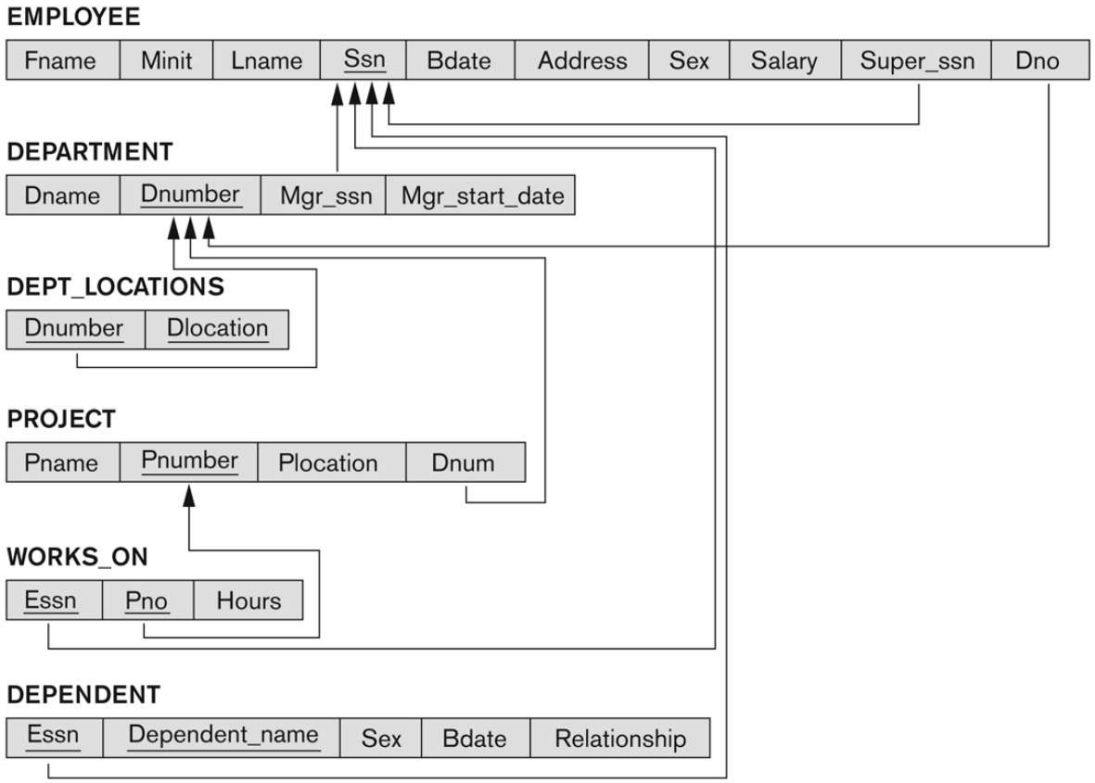
- Both work because this is inner join, in which order does not matter.
- Inner Join: returns only the rows that have matching values in both sides. If a row doesn't match, its excluded.

- If you want to retrieve non-matching rows as well, for example, if you want to retrieve all department names even if they don't handle any projects then we use Outer Join (left join, right join, full join)
- SELECT D.Dname, P.Pname FROM DEPARTMENT D LEFT JOIN PROJECT P ON P.Dnum = D.Dnumber;
- It will show all departments, even if they don't have any projects.
- Order is important in outer joins.

- Left Join: Returns all rows from the **left table** + matching rows from the right table.
- Right Join: Returns all rows from the **right table** + matching rows from the left table.
- Full Join: Returns all rows from both tables.
- Null values are used where there are no matches.

Question 2

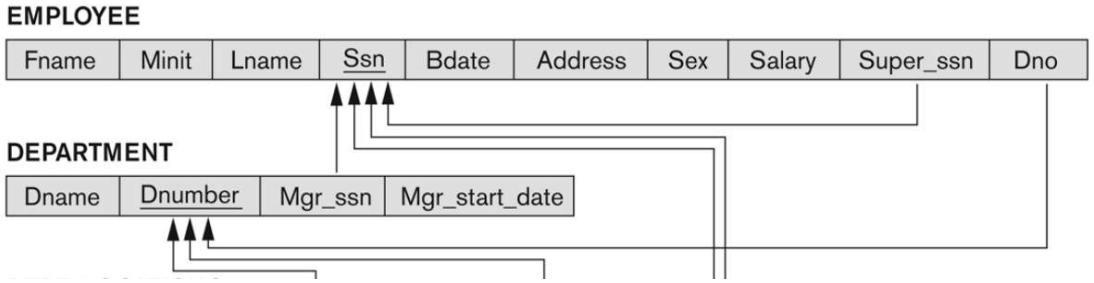
- Q2:Retrieve the name of the Employee and the name of projects they worked on?



Variations of JOIN

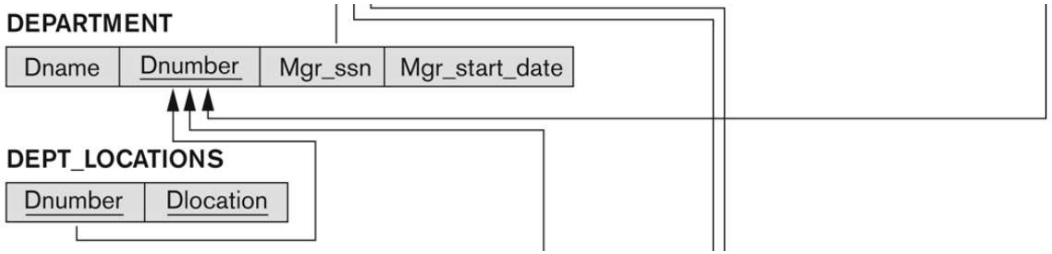
- **EQUIJOIN Operation**
  - The most common use of join involves join conditions with equality comparisons only
  - Such a join, where the only comparison operator used is =, is called an EQUIJOIN.

$$\text{DEPARTMENT} \bowtie_{\text{MGRSSN=SSN}} \text{EMPLOYEE}$$



Variations of JOIN

- **NATURAL JOIN Operation**
  - Requires that the two join attributes have the same name in both relations because it automatically matches on the column names.
  - NATURAL JOIN — denoted by *
  - Example: To apply a natural join on the DNUMBER attributes of DEPARTMENT and DEPT_LOCATIONS, it is sufficient to write:
  - DEPT_LOCS ← DEPARTMENT * DEPT_LOCATIONS



SELECT * FROM DEPARTMENT NATURAL JOIN DEPT_LOCATIONS;

Example

DEPT_LOCS

Dname	Dnumber	Mgr_ssn	Mgr_start_date	Location
Headquarters	1	888665555	1981-06-19	Houston
Administration	4	987654321	1995-01-01	Stafford
Research	5	333445555	1988-05-22	Bellaire
Research	5	333445555	1988-05-22	Sugarland
Research	5	333445555	1988-05-22	Houston

You don't need to explicitly write `ON DEPARTMENT.Dnumber = DEPT_LOCATIONS.Dnumber` because `NATURAL JOIN` handles that automatically.

Join with other queries

- **JOIN is not limited to SELECT**. While you'll see it most often in SELECT queries, you can also use JOINs in other SQL operations like INSERT, UPDATE and DELETE.

JOIN with UPDATE

- You can update rows in one table based on values in another table:
- UPDATE EMPLOYEE E JOIN DEPARTMENT D ON E.Dno = D.Dnumber SET E.Salary = E.Salary + 1000 WHERE D.Dname = 'Research';
- This increases the salary of employees in the Research department by 1000.

JOIN with INSERT

- INSERT INTO PROJECT_BACKUP SELECT P.Pname, D.Dname FROM PROJECT P JOIN DEPARTMENT D ON P.Dnum = D.Dnumber;
- The SELECT produces two columns:
  - P.Pname (project name)
  - D.Dname (department name)
- These values are inserted into PROJECT_BACKUP.
- The **target table** (PROJECT_BACKUP) must:
- Already exist.
- Have **exactly two columns**, *or* you must specify which columns you're inserting into.

Relational Algebra Operations from Set Theory: UNION

- UNION Operation
  - Binary operation, denoted by ∪
  - The result of R ∪ S, is a relation that includes all tuples that are either in R or in S or in both R and S
  - Duplicate tuples are eliminated
  - R and S must have same number of attributes
  - Corresponding attributes must be type compatible

Union Example

- **Example:**
  - To retrieve the social security numbers of all employees who either *work in department 5* (RESULT1 below) or *directly supervise an employee who works in department 5* (RESULT2 below)
  - We can use the UNION operation as follows:

$$\text{DEP5\_EMPS} \leftarrow \sigma_{\text{DNO=5}}(\text{EMPLOYEE})$$

$$\text{RESULT1} \leftarrow \pi_{\text{SSN}}(\text{DEP5\_EMPS})$$

$$\text{RESULT2} \leftarrow \pi_{\text{SUPERSSN}}(\text{DEP5\_EMPS})$$

$$\text{RESULT} \leftarrow \text{RESULT1} \cup \text{RESULT2}$$

  - The union operation produces the tuples that are in either RESULT1 or RESULT2 or both

Union Example

Figure 6.3
Result of the UNION operation RESULT ← RESULT1 ∪ RESULT2.

RESULT1

Ssn
123456789
333445555
666884444
453453453

RESULT2

Ssn
333445555
888665555

RESULT

Ssn
123456789
333445555
666884444
453453453
888665555

RESULT1: Employees who work in department 5

RESULT2: Employees who supervise an employee in department 5

SELECT Ssn FROM EMPLOYEE WHERE Dno = 5 UNION SELECT Super_ssn FROM EMPLOYEE WHERE Dno = 5 AND Super_ssn IS NOT NULL;

Exercise

- **Create 2 Tables:**
- Department (DeptID, DeptName);
- Employee (EmpID, EmpName, DeptID)
- **Insert Sample Data into both tables.**

Solve the following questions based on Join.

- **Inner Join**:
  List all employees with their department names.
- **Left Join**:
  List all departments and employees. Show employees as NULL if no one works in the department.
- **Right Join** (or use LEFT JOIN with swapped order):
  List all employees and their department names. Show department as NULL if not assigned.

Department table

DeptID	DeptName
1	HR
2	IT
3	Finance

Employee table

EmpID	EmpName	DeptID
101	Alice	1
102	Bob	2

DeptName	EmpName
HR	Alice
IT	Bob
Finance	NULL